Lab 4

Integrity Constraints, Sorting, Subqueries, and Data Filtering

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Sample Database Schema **5**

2 Integrity Constraints **6**
2.1 Primary Key 6
2.2 Foreign Key 7

3 Data Sorting (ORDER BY) **7**
3.1 Single Column Sorting 7
3.2 Multiple Column Sorting 7

4 Intermediate SQL: Subqueries **7**
4.1 Single-Row Subqueries 7
4.2 Multiple-Row Subqueries 8
4.3 Nested Subqueries (Multiple Nesting) 8
4.4 Correlated Subqueries 8

5 Range and Set Operators **9**
5.1 BETWEEN 9
5.2 DISTINCT 9

6 LIMIT **9**

7 Lab Tasks **10**

Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

1. Connect to Your Database:

Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Verify the Connection:

Verify that you are connected to the correct database with the following command:

```
\conninfo
```

Expected output:

```
Database: cse4308_12345, User: student_12345
```

3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

Load the SQL file that sets up the tables and inserts initial data.

1. Load the SQL File:

Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_04_setup.sql
```

This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Check Tables:

Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L4_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

1 Sample Database Schema

To demonstrate the concepts in this lab, we will refer to the following three tables: **Department**, **Employee**, and **Project**.

Table 1: Department

dept_id (PK)	**dept_name**	**location**
D01	CSE	Building A
D02	EEE	Building B
D03	BBA	Building C

Table 2: Employee

emp_id (PK)	**name**	**salary**	**dept_id** (FK)
E101	Alice	50000	D01
E102	Bob	60000	D01
E103	Charlie	55000	D02
E104	David	45000	D03
E105	Eve	70000	D01

Table 3: Project

proj_id (PK)	**proj_name**	**dept_id** (FK)
P01	AI Research	D01
P02	Circuit Design	D02
P03	Web Dev	D01

```
-- 1. Clean up existing tables (if any) to avoid conflicts
-- We use CASCADE to remove foreign key dependencies automatically
DROP TABLE IF EXISTS Project CASCADE;
DROP TABLE IF EXISTS Employee CASCADE;
DROP TABLE IF EXISTS Department CASCADE;

-- 2. Create the Department Table (Parent Table)
CREATE TABLE Department (
    dept_id VARCHAR(5),
    dept_name VARCHAR(20),
    location VARCHAR(50),
    CONSTRAINT pk_department PRIMARY KEY (dept_id)
);

-- 3. Create the Employee Table (Child Table)
CREATE TABLE Employee (
    emp_id VARCHAR(5),
```

```
    name VARCHAR(30),
    salary NUMERIC(10, 2),
    dept_id VARCHAR(5),
    CONSTRAINT pk_employee PRIMARY KEY (emp_id),
    CONSTRAINT fk_emp_dept FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- 4. Create the Project Table (Child Table)
CREATE TABLE Project (
    proj_id VARCHAR(5),
    proj_name VARCHAR(50),
    dept_id VARCHAR(5),
    CONSTRAINT pk_project PRIMARY KEY (proj_id),
    CONSTRAINT fk_proj_dept FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- 5. Insert Data into Department
-- (Must be done first because other tables reference dept_id)
INSERT INTO Department (dept_id, dept_name, location) VALUES
('D01', 'CSE', 'Building A'),
('D02', 'EEE', 'Building B'),
('D03', 'BBA', 'Building C');

-- 6. Insert Data into Employee
INSERT INTO Employee (emp_id, name, salary, dept_id) VALUES
('E101', 'Alice', 50000, 'D01'),
('E102', 'Bob', 60000, 'D01'),
('E103', 'Charlie', 55000, 'D02'),
('E104', 'David', 45000, 'D03'),
('E105', 'Eve', 70000, 'D01');

-- 7. Insert Data into Project
INSERT INTO Project (proj_id, proj_name, dept_id) VALUES
('P01', 'AI Research', 'D01'),
('P02', 'Circuit Design', 'D02'),
('P03', 'Web Dev', 'D01');
```

2 Integrity Constraints

2.1 Primary Key

A Primary Key is a column (or a set of columns) that uniquely identifies each row in a table. It must contain unique values and cannot contain NULL values.

```
CREATE TABLE Department (
    dept_id VARCHAR(5),
    dept_name VARCHAR(20),
    location VARCHAR(50),
    CONSTRAINT pk_department PRIMARY KEY (dept_id)
);
```

2.2 Foreign Key

Foreign keys are used to enforce referential integrity between two tables. A foreign key in one table references the primary key of another table.

```
CREATE TABLE Employee (
    emp_id VARCHAR(5),
    name VARCHAR(30),
    salary NUMERIC(10, 2),
    dept_id VARCHAR(5),
    CONSTRAINT pk_employee PRIMARY KEY (emp_id),
    CONSTRAINT fk_emp_dept FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Note: The `ON DELETE CASCADE` clause ensures that if a Department is deleted, all Employees belonging to that department are also automatically deleted.

3 Data Sorting (ORDER BY)

The `ORDER BY` clause is used to sort the result set in either ascending (ASC) or descending (DESC) order.

3.1 Single Column Sorting

Retrieve employees sorted by salary in descending order.

```
SELECT name, salary
FROM Employee
ORDER BY salary DESC;
```

3.2 Multiple Column Sorting

Sort by department ID first, then by salary within that department.

```
SELECT dept_id, name, salary
FROM Employee
ORDER BY dept_id ASC, salary DESC;
```

4 Intermediate SQL: Subqueries

A subquery (also known as a nested query or inner query) is a query within another SQL query.

4.1 Single-Row Subqueries

These subqueries return a single value (one row, one column). They are typically used with comparison operators ($=, >, <, >=, <=, <>$).

Example: Find employees who earn more than the average salary of all employees.

```sql
SELECT name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

4.2 Multiple-Row Subqueries

These subqueries return more than one row. They are used with set operators like `IN`, `ANY`, or `ALL`.

Example (IN): Find employees who work in departments located in `'Building A'`.

```sql
SELECT name
FROM Employee
WHERE dept_id IN (
    SELECT dept_id
    FROM Department
    WHERE location = 'Building A'
);
```

4.3 Nested Subqueries (Multiple Nesting)

SQL allows nesting subqueries inside other subqueries.

Example: Find the name of employees who work in a department that has an active project named `'AI Research'`.

Logic: Find the project's department → Find that department in the Employee table.

```sql
SELECT name
FROM Employee
WHERE dept_id IN (
    SELECT dept_id
    FROM Project
    WHERE proj_name = 'AI Research'
);
```

You can also use "NOT IN" to find the people who are not in those results **Example (Deep Nesting):** Find employees who work in a department where the department ID exists in the Project table, but only for projects managed by the `'CSE'` department.

```sql
SELECT name
FROM Employee
WHERE dept_id IN (
    SELECT dept_id
    FROM Project
    WHERE dept_id = (
        SELECT dept_id
        FROM Department
        WHERE dept_name = 'CSE'
    )
);
```

4.4 Correlated Subqueries

A correlated subquery depends on the outer query for its values. It is executed once for *each row* selected by the outer query.

Example: Find employees whose salary is higher than the average salary **of their specific department**.

```
SELECT e1.name, e1.salary, e1.dept_id
FROM Employee e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e1.dept_id
);
```

5 Range and Set Operators

5.1 BETWEEN

```
SELECT name, salary
FROM Employee
WHERE salary BETWEEN 45000 AND 60000;
```

5.2 DISTINCT

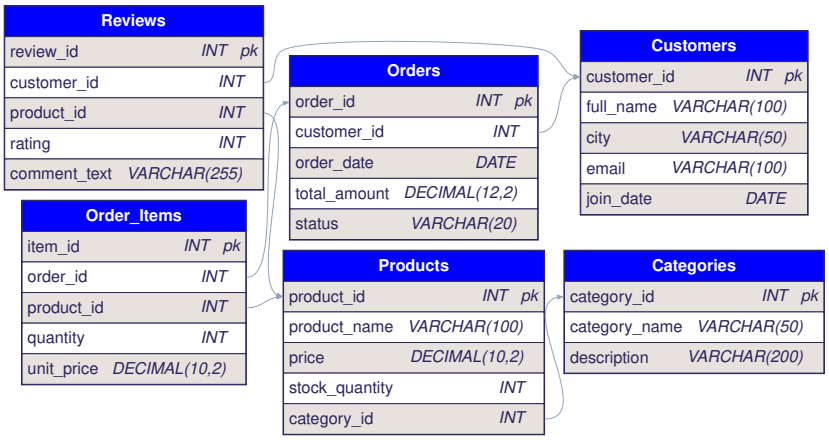
```
-- Returns unique department IDs from the Employee table
SELECT DISTINCT dept_id FROM Employee;
```

6 LIMIT

Used for pagination or restricting results.

```
-- Top 3 highest paid employees
SELECT name, salary
FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

7 Lab Tasks



1. Retrieve the names and prices of all products, sorted by price from highest to lowest.
2. List all customers sorted alphabetically by their full name.
3. Find the top 5 most expensive orders placed based on `total_amount`.
4. List the `product_name` and `stock_quantity` of products, sorted first by `category_id` (ascending) and then by `stock_quantity` (descending).
5. Retrieve the 3 most recent orders (based on `order_date`).
6. List all reviews sorted by rating (lowest first); if ratings are equal, sort by `review_id` descending.
7. Find the names of products with a price between 50 and 200 (inclusive).
8. Retrieve unique city names where your customers reside.
9. Find all orders placed between `'2023-06-01'` and `'2023-12-31'`.
10. List the names of customers who live in `'New York'`, `'Chicago'`, or `'Miami'`.
11. Find products that belong to category IDs 1, 3, or 5 using the `IN` operator.
12. Retrieve unique `product_id`s that have received a review.
13. Find the products that are more expensive than the average price of all products.
14. Retrieve the customer(s) who placed the specific order with the highest `total_amount` in the database.
15. List products that have less stock than the average stock quantity of all products.
16. Find the name of the product that has the lowest price in the entire inventory.
17. Find orders where the `total_amount` is greater than the average total amount of all orders.

18. Retrieve the name of the customer who has the earliest `join_date` (i.e., the first customer to join).

19. Find the names of products that belong to the `'Furniture'` category (using a subquery on the `Categories` table).

20. Find the full names of customers who have placed at least one order (check if `customer_id` is in the `Orders` table).

21. Find the names of products that have **never** been ordered (product ID not in `Order_Items`).

22. **Deep Nesting:** Find the names of customers who have ordered a product from the `'Smartphones'` category.

23. Find the `review_id` and `comment_text` for reviews written by customers from `'New York'`.

24. Find products that have been ordered in a quantity greater than 2 in a single line item (using `Order_Items`).

25. Find products that are more expensive than the average price **of their specific category**.

26. **Correlated:** Find customers who have placed an order but have **never** written a review.

27. Find customers who have ordered the product named `'Dell XPS 13'`.

28. Find products that have a rating lower than the average rating of all reviews.

29. **Correlated:** Find orders where the `total_amount` is larger than the average total amount of orders placed by that specific customer.